Kaggle Competition

By Nikita Park, Jan Turner,
Mingchen Wang, Jeremy Shiu,
and Josh Xu

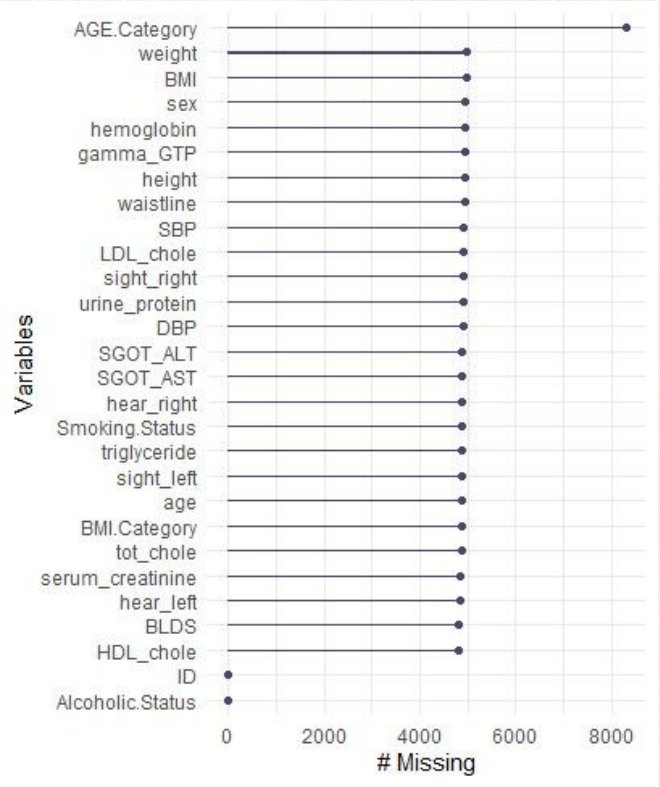
Project Timeline

- **11/23 – 11/26:** Impute thousands of data points in both the training and testing datasets, as most classifiers considered do not allow for missing values
- **11/26 – 11/29:** Consider classification methods such as LDA, QDA, KNN, XGBoost, GLM, and randomForest, along with predictor modifications such as PCA, PCR, Reclassification, and variable transformations
- **11/30 – 12/8:** Fine tune our best model by investigating which variables are the best predictors and adjusting parameters to maximize testing accuracy
- **5th place finish!**

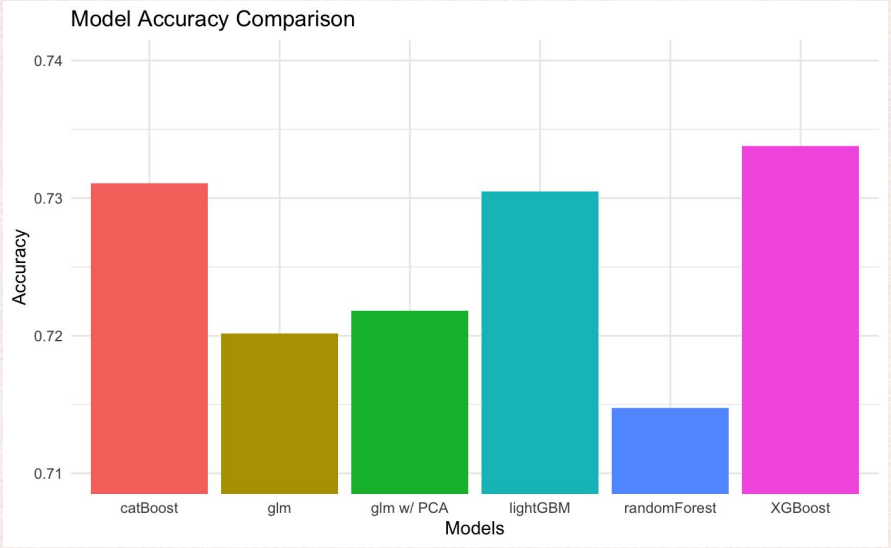
Filling in NA Values

For the training data, there were over 130,000 missing values, and for the testing, over 56,000 missing values.

Methods we tried include MICE, aregImpute, missForest, and Amelia. The aregImpute method emerged as the winning method for us.



Types of models we tried



- We tried a few other models like KNN, LDA and QDA, but the training accuracies were too subpar.

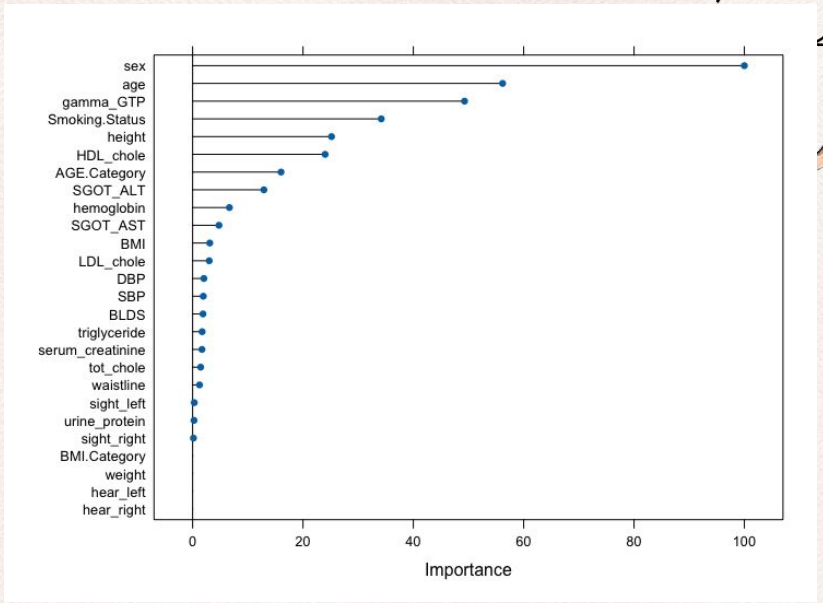
Our best submission had an accuracy score of 0.7338

- The best submission used Multiple Imputation using Additive regression, Bootstrapping, and Predictive Mean Matching (aregImpute) for the imputation technique, with parameters n.impute = 5 and otherwise default settings
- Despite concerns of overfitting, using all the variables ended up producing the best results in terms of testing accuracy. The best model we tested was the XGBoost model using method “xgbtree”
- Additional methods we used to make miniscule increases in accuracy included trying different odd seeds, and running different parameters in the imputation and model creation.

Variable selection

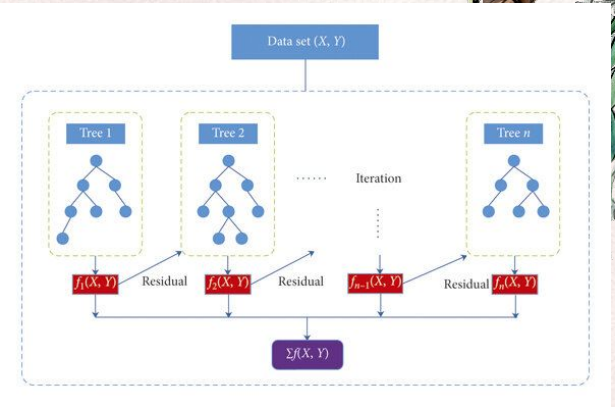
In XGBoost, "importance" is a measure of the improvement in the model's decision tree using a singular predictor to split the data. A predictor's importance is calculated by summing up the improved accuracies on the decision tree by using that predictor to split the data. So, the higher the importance of the predictor, the higher the importance.

Nevertheless, using all the variables tended to squeeze out the best results.



How our Final Model works

- Our final model combined the use of bootstrapping and XGBoost.
  - XGBoost - Short for eXtreme Gradient Boosting
  - Bootstrapping - Used for resampling train data for multiple trainings, to reduce overfitting and estimate best tune parameters.



- XGBTree is a XGBoost method that builds decision trees models sequentially, with each trees reducing the error of previous ones. These trees are combined and used to develop a final predictive model once no further errors can be reduced using the algorithm.

Code for the final model:

```
train(as.factor(Alcoholic.Status) ~ ., data = xgboost_data, method = "xgbTree", trControl = trainControl("boot", number = 10), verbosity = 0, tuneLength = 2)
```

references

- Han, Yixiu, et al. “Coupling a Bat Algorithm with XGBoost to Estimate Reference Evapotranspiration in the Arid and Semiarid Regions of China.” *Advances in Meteorology*, Hindawi, 17 Oct. 2019, www.hindawi.com/journals/amete/2019/9575782/.
- “Introduction to Boosted Trees.” *Introduction to Boosted Trees - Xgboost 2.0.2 Documentation*, xgboost developers, 2022, xgboost.readthedocs.io/en/stable/tutorials/model.html.
- “What Is XGBoost?” *NVIDIA Data Science Glossary*, NVIDIA, www.nvidia.com/en-us/glossary/data-science/xgboost/. Accessed 9 Dec. 2023.